

Ссылка на задание на реплит:

<https://replit.com/@evgeniiayd/lablonskaia-levghienii-IVT-12-1-kurs-LR-5#>

Комплект 1: Многофайловый проект, условная компиляция, утилита Make.

1. Постановка задачи.

Напишите программу из нескольких файлов (модулей), включая файл основной программы. Файлы должны содержать вынесенные отдельно функции для выделения памяти под динамические двумерные и одномерные массивы и функции для перемножения матриц. Собрать проект используя утилиту Make.

2. Математическая модель.

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{pmatrix} =$$
$$= \begin{pmatrix} a_{11} \cdot b_{11} + a_{12} \cdot b_{21} & a_{11} \cdot b_{12} + a_{12} \cdot b_{22} & a_{11} \cdot b_{13} + a_{12} \cdot b_{23} \\ a_{21} \cdot b_{11} + a_{22} \cdot b_{21} & a_{21} \cdot b_{12} + a_{22} \cdot b_{22} & a_{21} \cdot b_{13} + a_{22} \cdot b_{23} \\ a_{31} \cdot b_{11} + a_{32} \cdot b_{21} & a_{31} \cdot b_{12} + a_{32} \cdot b_{22} & a_{31} \cdot b_{13} + a_{32} \cdot b_{23} \end{pmatrix}$$

3. Список идентификаторов.

Имя	Тип	Смысл
n	int	Входная переменная
*b	double	Указатель на входной массив
i	int	Счётчик цикла
t	int	Входная переменная
k	int	Входная переменная
m	int	Входная переменная
r	int	Входная переменная
A	double**	Указатель на массив входных значений

C	double**	Указатель на массив входных значений
E	double**	Результат
j	int	Счётчик цикла
*allocate_array	double	Функция выделения памяти под массив
*free_array	double	Функция освобождения памяти, выделенной под массив
allocate_matrix	double**	Функция выделения памяти под матрицу
M	const int	Параметр функции allocate_matrix
N	const int	Параметр функции allocate_matrix
*ptr	void	Переменная функции allocate_matrix
**A	double	Переменная функции allocate_matrix
*A_date	double	Переменная функции allocate_matrix
**multi	double	Функция перемножения матриц
D	double**	Параметр функции multi
K	double**	Параметр функции multi
f	int	Параметр функции multi
g	int	Параметр функции multi
h	int	Параметр функции multi
l	int	Параметр функции multi
s	double	Переменная функции multi
E	double**	Переменная функции multi

4. Код программы.

makefile

```
1  TARGET = Project
2  CC = gcc
3
4  PREF_SRC = ./src/
5  PREF_OBJ = ./obj/
6
7  SRC = $(wildcard $(PREF_SRC)*.c)
8  OBJ = $(patsubst $(PREF_SRC)%.c, $(PREF_OBJ)%.o, $(SRC))
9
10 $(TARGET) : $(OBJ)
11     $(CC) $(OBJ) -o $(TARGET)
12
13 $(PREF_OBJ)%.o : $(PREF_SRC)%.c
14     $(CC) -c $< -o $@
15
16 clean :
17     del $(TARGET).exe obj\*.o
18
```

main.c:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "array.h"
4  #include "matrix.h"
5  #include "multi.h"
6
7  int main()
8  {
9      int n;
10     printf("Enter n\n");
11     scanf("%d", &n);
12
13     double *b = allocate_array(n);
14
15     for(int i = 0; i < 3; i++) {
16         scanf("%d", &b[i]);
17     }
18
19     free_array(b);
20
21     int t = 2;
22     int k = 3;
23     int m = 3;
24     int r = 1;
25
26     double** A = allocate_matrix(t, k);
27     double** C = allocate_matrix(m, r);
28
29     A[0][0] = 1;
```

```

29     A[0][0] = 1;
30     A[0][1] = 2;
31     A[0][2] = 3;
32     A[1][0] = 1;
33     A[1][1] = 1;
34     A[1][2] = 1;
35
36     C[0][0] = 0;
37     C[1][0] = 1;
38     C[2][0] = 3;
39
40     double** E = multi(A, t, k, C, m, r);
41
42     printf("Answer:\n ");
43     for(int i = 0; i < t; ++i) {
44         for(int j = 0; j < r - 1; ++j) {
45             printf("%f, ", E[i][j]);
46         }
47         printf("%f\n", E[i][r - 1]);
48     }
49
50     free(A);
51     free(C);
52
53     return 0;
54 }
55

```

array.c:

```

1  #include <stdlib.h>
2
3  double* allocate_array(int n) {
4      double *b = malloc (n * sizeof (double *));
5      return b;
6  };
7
8  double* free_array(double* b) {
9      free(b);
10 };
11

```

array.h:

```

1  #ifndef ARRAY_H
2  #define ARRAY_H
3
4  double* allocate_array(int n);
5
6  double* free_array(double* b);
7
8  #endif // ARRAY_H
9

```

matrix.c:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  double** allocate_matrix(const int M, const int N) {
5      void *ptr = malloc (M * sizeof (double *) + M * N * sizeof (double));
6      double **A = (double **) ptr;
7      double *A_data = (double *) (A + M);
8
9      for (int i = 0; i < M; i++) {
10         A[i] = A_data + i*N;
11     }
12
13     return A;
14 }
15
```

matrix.h:

```
1  #ifndef MATRIX_H
2  #define MATRIX_H
3
4  double** allocate_matrix(const int M, const int N);
5
6  #endif // MATRIX_H
7
```

multi.c:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "matrix.h"
4
5  double** multi(double** D, int f, int g, double** K, int h, int l) {
6      double** E = allocate_matrix(f,l);
7      double s;
8      for(int i = 0; i < f; i++) {
9          for (int j = 0; j < l; j++) {
10             s = 0;
11             for(int o = 0; o < g; o++) {
12                 s = s + D[i][o]*K[o][j];
13             }
14             E[i][j] = s;
15         }
16     }
17     return E;
18 }
19
```

multi.h:

```
1  |#ifndef MULTI_H
2  |#define MULTI_H
3
4  |double** multi(double** D, int f, int g, double** K, int h, int l);
5
6  |#endif
7
```

5. Результат выполнения программы.

```
PS C:\Users\werrr\Desktop\study\Программирование\ЛР 5\MakeFileExample> make
gcc -c src/array.c -o obj/array.o
gcc -c src/main.c -o obj/main.o
gcc -c src/matrix.c -o obj/matrix.o
gcc -c src/multi.c -o obj/multi.o
gcc ./obj/array.o ./obj/main.o ./obj/matrix.o ./obj/multi.o -o Project
PS C:\Users\werrr\Desktop\study\Программирование\ЛР 5\MakeFileExample> ./Project
Enter n
3
1
2
3
Answer:
11.000000
4.000000
PS C:\Users\werrr\Desktop\study\Программирование\ЛР 5\MakeFileExample>
```